

Skills in Claude & Cursor

A beginner-friendly guide to one of the most powerful features of modern AI tools. Skills let you teach an AI assistant *how you want it to work* — once — and then trigger that behaviour with a single word.

Who is this for? This guide assumes no prior AI engineering experience. If you have ever wished your AI assistant "just knew" how to format a document or write code the way you like — skills are the answer.

#	Section	What you will learn
1	What is a Skill?	Definition, analogy, and core idea
2	Why Are Skills Needed?	The problem they solve
3	Without Skills vs With Skills	Side-by-side comparison with diagrams
4	How Skills Work	The trigger-and-execute workflow
5	Skill Examples	Real-world examples for Claude & Cursor
6	Creating a Skill in Claude	Step-by-step guide with code
7	Creating a Skill in Cursor	Step-by-step guide with code
8	Claude vs Cursor — Comparison	Key differences and when to use each

1. What Is a Skill?

A **skill** is a saved set of instructions that tells an AI assistant exactly how to perform a specific task. Instead of you typing a long, detailed prompt every time, you write those instructions once, save them as a skill, and from then on you just say the skill's name. The AI does the rest.

Real-world analogy: Think of a skill like a recipe card in a professional kitchen. Instead of the chef explaining from scratch how to make Bolognese every service, they write it on a recipe card once. Any chef can then follow that card and produce the exact same dish every time — fast and consistent.

Skills in two tools

Tool	What a 'Skill' looks like	Purpose
Claude	A SKILL.md file with instructions	Extend Claude's task capabilities (create Word docs, PDFs, etc.)
Cursor	A .mdc rule file in .cursor/rules/	Guide Cursor's AI on coding style, patterns, and standards

The three things every skill does

① Defines the task

Tells the AI what kind of work to do — e.g. "create a Word document" or "write Python using type hints".

② Provides detailed instructions

Supplies all the how-to knowledge once, so you never have to repeat it — fonts, formatting, code style, structure, etc.

③ Sets a trigger

Gives the AI a signal to load those instructions — a keyword, a file type, or a description that matches your request.

2. Why Are Skills Needed?

Without skills, every single conversation with an AI starts from scratch. You have to re-explain your requirements, formatting preferences, and standards every time. This is slow, inconsistent, and error-prone.

The core problem

Pain point	Without Skills	With Skills
Time	Type a 200-word prompt every task	Type 3 words — done
Consistency	Output varies each time you ask	Same quality output every time
Knowledge sharing	Each person writes their own prompts	Share a skill file — everyone benefits
Errors	Easy to forget a formatting rule	Instructions are locked in the skill
Onboarding	New teammates need to learn prompts	Install the skill — they're ready

What skills enable

- **Reusability** — Write instructions once, use them hundreds of times.
- **Consistency** — Every output follows the same rules, whoever triggers the skill.
- **Speed** — No more typing long prompts. A single command is enough.
- **Teamwork** — Skills are shareable files. Your whole team can use the same standards.
- **Specialisation** — Build deep expertise into a skill once; the AI carries it forever.

3. Without Skills vs With Skills

The diagrams below show the difference in a single concrete scenario: three people on a team each need to create a Word document.

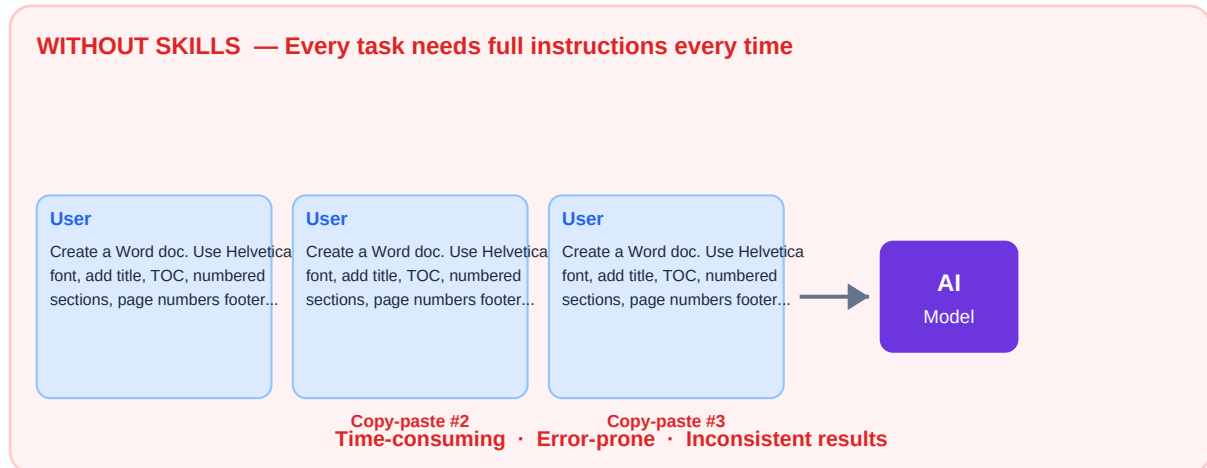


Figure 1 — Without skills: every user must type the full prompt from memory. Results vary. Time is wasted.

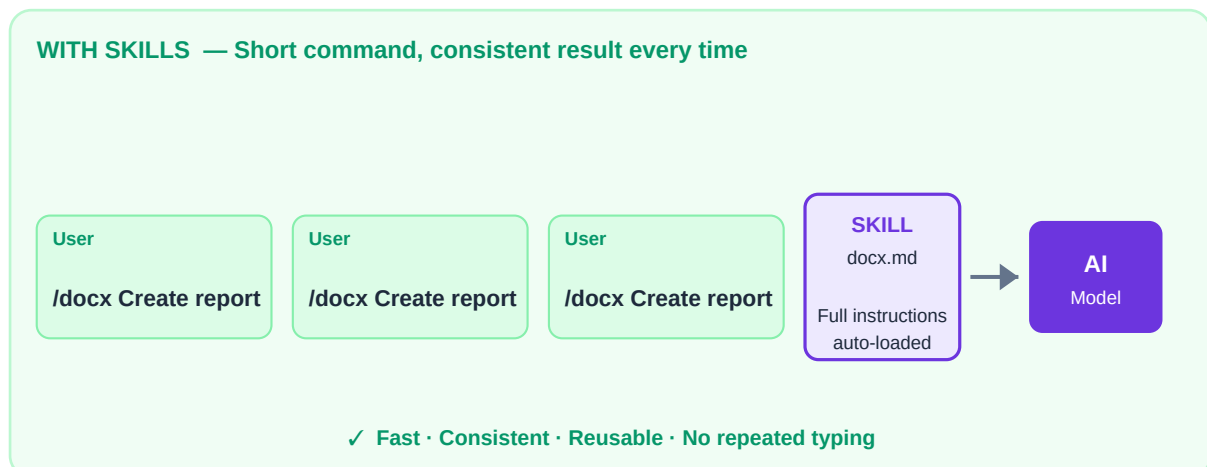


Figure 2 — With skills: one short command triggers the full instructions automatically. Consistent, fast, shareable.

Key insight: A skill moves the complexity from the user's memory to a file. Once the file exists, the user only needs to say the skill's name. The AI knows the rest.

4. How Skills Work

Behind the scenes, skills follow a simple trigger-and-execute pattern. When you send a message, the AI checks whether any installed skill matches your request. If it does, the skill's instructions are loaded — invisibly — before the AI responds.



Figure 3 — The five-step skill execution flow from user command to delivered output.

What happens at each step

Step	What happens	Who does it
1 — Command	You type a short message or command	You
2 — Detection	The AI reads the skill's trigger description and decides if it applies	AI (automatic)
3 — Load	The SKILL.md (or .mdc) file is read by the AI	System (automatic)
4 — Execute	The AI follows the skill's instructions to do your task	AI
5 — Output	The finished result is returned to you	AI

Skill triggers — how the AI knows which skill to use

Each skill has a **description** that tells the AI when to use it. The AI reads your message and matches it against all available skill descriptions.

Trigger type	Example	Used in
Task description	"create a Word document" → loads docx skill	Claude
Slash command	"/docx report" → loads docx skill	Claude

File type (glob)	<code>"*.py"</code> → loads python coding rules	Cursor
Explicit mention	<code>"@python-rules"</code> → loads that rule file	Cursor
Always-on	Rule applies to every file in the project	Cursor

5. Real-World Examples

Claude Skill Examples

Skill name	What it does	Trigger
docx	Creates formatted Word documents with correct styles, headers, TOC, and page numbers	User asks to create a Word/doc file
xlsx	Builds Excel spreadsheets with formulas, formatting, and charts	User asks for a spreadsheet or .xlsx
pdf	Generates polished PDF reports with custom layout	User asks for a PDF output
pptx	Builds PowerPoint slide decks from content	User asks for a presentation or slides
schedule	Creates a scheduled task that runs automatically	User says 'every morning', 'remind me'
skill-creator	Helps the user build new skills	User asks to create or edit a skill

Cursor Rule Examples

Rule file	What it does	Applies to
python.mdc	Enforces type hints, docstrings, PEP-8 style	All *.py files
react.mdc	Uses functional components, hooks, Tailwind CSS	*.tsx / *.jsx files
api-design.mdc	REST API naming, versioning, error-handling rules	Routes / controller files
testing.mdc	Requires pytest, AAA pattern, 80%+ coverage	test_*.py files
git-commits.mdc	Conventional commit message format	Always on
docs.mdc	Docstring format, README structure standards	*.md and *.py files

Example: Claude skill in action

Claude — docx skill triggered automatically

```
# User types just this:  
Create a project status report for the Q3 launch.  
  
# Claude detects: 'report' → loads docx skill automatically  
# Skill tells Claude: use Helvetica 11pt, add TOC, numbered  
# sections, page numbers, logo placeholder, executive summary...  
  
# Claude produces a fully formatted .docx — no extra instructions needed.
```

Example: Cursor rule in action

Cursor — python.mdc rule triggered automatically

```
# User opens user_service.py and asks Cursor:  
Add a method to get a user by email address.  
  
# Cursor detects: *.py file → loads python.mdc rule  
# Rule says: use type hints, write docstring, raise ValueError not None  
  
# Cursor generates:  
def get_user_by_email(self, email: str) -> User:  
    """Retrieve a user by their email address.  
  
    Args:  
        email: The user's email address.  
  
    Returns:  
        The matching User object.  
  
    Raises:  
        ValueError: If no user with that email exists.  
    """  
    user = self.db.query(User).filter_by(email=email).first()  
    if not user:  
        raise ValueError(f"No user found with email: {email}")  
    return user
```


6. Creating a Skill in Claude

A Claude skill is a folder containing a single **SKILL.md** file. That file has two parts: a header (metadata) and a body (the instructions). Claude reads this file whenever the skill is triggered.

Step 1 — Understand the SKILL.md structure

SKILL.md — general structure

My Skill Name

<!-- This comment block is the metadata header -->

<!--

description: >

One or two sentences describing WHEN this skill should be used.

Claude reads this to decide if the skill applies to a user's request.

-->

What this skill does

Plain English explanation of the task.

Instructions

Step-by-step instructions for Claude to follow.

Output format

Describe exactly what the output should look like.

Step 2 — Write a real SKILL.md (example: meeting notes)

SKILL.md — meeting notes example

Meeting Notes Formatter

<!--

description: >

Use this skill when the user wants to format, clean up, or structure meeting notes. Triggers on: 'meeting notes', 'format notes', 'write up the meeting', 'meeting summary'.

-->

What this skill does

Converts raw meeting notes into a clean, structured document.

Instructions

1. Extract the meeting date, attendees, and agenda from the input.
2. Group discussion points under clear headings.
3. List all action items with owner and due date.
4. Write an executive summary (3 sentences max) at the top.
5. Use bullet points for decisions made.

Output format

- Title: Meeting Notes – [Date]
- Section: Executive Summary
- Section: Attendees
- Section: Discussion (grouped by topic)
- Section: Decisions Made
- Section: Action Items (table: Task | Owner | Due Date)

Step 3 — Set up the folder structure

Each skill lives in its own folder. The folder name is the skill name. Place all skill folders inside a parent **skills/** directory.

```
skills/  
  docx/  
    SKILL.md  
  xlsx/  
    SKILL.md  
  pdf/  
    SKILL.md
```

Your skills folder structure:

- Each subfolder = one skill
- Each skill has exactly one SKILL.md
- Skill name = folder name
- You can have as many skills as you like

The **skills/** folder can live anywhere on your computer. You tell Claude where it

Step 4 — Install the skill in Claude

Step	Action
1	Open Claude desktop app
2	Go to Settings → Capabilities
3	Click 'Add Skills Folder' and point it to your skills/ directory
4	Claude will automatically detect all SKILL.md files in subfolders
5	Test it: ask Claude to do the task — it should trigger the skill automatically

Tip: After installing, you can say something like *"format my meeting notes from today"* and Claude will automatically detect and apply your Meeting Notes skill — no slash commands needed.

7. Creating a Skill (Rule) in Cursor

In Cursor, skills are called **rules**. They live in a `.cursor/rules/` folder at the root of your project. Each rule is a `.mdc` file (Markdown with a YAML-like header). Cursor reads the relevant rule whenever you open a matching file or ask for help.

Step 1 — Understand the `.mdc` file structure

.cursor/rules/python.mdc — structure

```
---
description: One sentence – when should Cursor apply this rule?
globs: ['*.py', 'tests/**/*.py'] # which files trigger this rule
alwaysApply: false # true = apply to every file
---

# Rule Title

## Coding standards
- Always use type hints on function signatures
- Write docstrings in Google style
- Raise exceptions; never return None for errors

## Naming conventions
- snake_case for variables and functions
- PascalCase for classes
- UPPER_SNAKE_CASE for constants

## What to avoid
- No bare `except:` clauses
- No mutable default arguments
- No `print()` – use logging instead
```

Step 2 — Create your first rule file

.cursor/rules/react.mdc — full example

```
---
description: >
  Apply when working on React components. Enforces functional
  components, custom hooks, and Tailwind CSS for styling.
  globs: ['src/**/*.tsx', 'src/**/*.jsx', 'components/**/*.tsx']
  alwaysApply: false
---

# React Component Standards

## Component rules
- Use functional components only (no class components)
- Keep components under 150 lines; split if larger
- Props must be typed with a TypeScript interface

## Styling
- Use Tailwind CSS utility classes only
- No inline styles or CSS-in-JS

## State management
- Use useState for local state
- Extract complex logic into custom hooks (use prefix)
- Avoid prop drilling beyond 2 levels; use Context instead

## File structure per component
- types.ts (interfaces)
- index.tsx (component)
- hooks.ts (custom hooks)
- utils.ts (helper functions)
```

Step 3 — Set up the folder structure

```
.cursor/
├── rules/
│   ├── python.mdc
│   ├── react.mdc
│   └── docs.mdc
└── .cursorrules
```

Your project structure:

- `.cursor/rules/` = where all rules live
- Each `.mdc` file = one rule / skill
- globs = which files activate it
- `.cursorrules` = legacy (still works)

*The `.cursor/` folder goes at the root of each project.
Commit it to git so your whole team shares the same rules.*

Step 4 — Activate and use your rule in Cursor

Step	Action
1	Create the <code>.cursor/rules/</code> folder at your project root
2	Add your <code>.mdc</code> rule files inside it
3	Open Cursor — it auto-detects rules in that folder
4	Open a file matching your glob pattern (e.g. <code>App.tsx</code>)
5	Ask Cursor AI to help — it will follow your rule automatically
6	You can also @mention a rule: type <code>@react</code> to force it

Cursor — rule applied automatically

In the Cursor chat panel, just ask:

Add a search bar component to the dashboard.

Cursor detects: `Dashboard.tsx` → loads `react.mdc` rule

Rule enforces: functional component, TypeScript interface,

Tailwind CSS, extract hook if stateful

Result: properly structured component following your team standards

— without you having to mention the rules at all.

8. Claude vs Cursor — Side-by-Side

Claude (Cowork)	Cursor (AI Code Editor)
• Skills = SKILL.md files • Installed via Settings > Capabilities • Triggered by task type / description • Extends Claude's output capabilities • Examples: docx, xlsx, pdf, pptx	• Skills = .mdc rule files • Stored in .cursor/rules/ folder • Triggered by file type or @mention • Guides AI coding style & standards • Examples: python.mdc, react.mdc

Figure 4 — Claude skills extend what Claude produces; Cursor rules guide how Cursor's AI writes code.

Feature	Claude Skills	Cursor Rules
File type	SKILL.md	.mdc (or .cursorrules)
File location	Skills folder (any location)	.cursor/rules/ in project root
Trigger mechanism	Task description matching	File glob pattern or @mention
Primary use	Output format & task capabilities	Code style & engineering standards
Sharing	.skill bundle (zip) via Settings	Commit .cursor/ folder to git
Scope	Cross-project (installed globally)	Per-project (in repo)
Complexity	Simple markdown instructions	YAML header + markdown body
Best for	Document creation, automation tasks	Code quality, team standards

When to use which?

Use Claude Skills when you want to save how Claude should produce something — a document, a PDF, a scheduled task, a presentation. The skill controls the *output*.

Use Cursor Rules when you want to control how Cursor's AI writes code in your project — naming conventions, testing requirements, forbidden patterns. The rule controls the *coding process*.

Use both together for maximum power: Cursor writes high-quality code following your rules, and Claude turns that work into formatted reports, documentation, or presentations using its skills.

Quick start checklist

✓	Action
■	Write a SKILL.md for a task you repeat often in Claude
■	Install it via Claude Settings → Capabilities
■	Test: ask Claude to do that task — confirm the skill triggers
■	Create a .cursor/rules/ folder in your next code project
■	Write one .mdc rule for your primary language (Python/JS/etc.)
■	Ask Cursor AI to write a function — check it follows your rule
■	Share the .cursor/ folder with your team via git